

ISSE

Sistema de Certificación Docente

FICHA TÉCNICA — DOCUMENTACIÓN INTEGRAL DEL PROYECTO

Plataforma web full-stack de gestión y certificación de carga horaria docente

Universidad de La Sabana · Facultad de Ingeniería

Incluye: Plataforma Web · Pipeline ETL · Bot Telegram AVI Jr · Neon PostgreSQL

AVI Jr | Bot Telegram + GPT-4o-mini | 24/7

Resumen Ejecutivo

ISSE (Sistema de Certificación Docente) es una plataforma tecnológica integral desarrollada para la Facultad de Ingeniería de la Universidad de La Sabana, diseñada para transformar radicalmente la forma en que se gestiona, consulta y certifica la carga horaria de los docentes. El sistema comprende tres componentes principales: una **interfaz web moderna** construida con React 19 y TypeScript, un **backend REST de alto rendimiento** sobre FastAPI y PostgreSQL, y el **Bot Telegram AVI JR**, un agente inteligente disponible las 24 horas del día que pone la información certificada al alcance de cualquier dispositivo con Telegram instalado.

El proyecto abarca **10 años de historia académica** (2016–2026), con **555 profesores** distribuidos en **20 períodos académicos**. El archivo fuente del SIS universitario contiene muchas más filas brutas, pero tras el proceso de limpieza del ETL — eliminación de duplicados exactos, deduplicación de secciones combinadas y fusión de cursos idénticos — el sistema produce **6,603 registros certificados limpios**. Todo el ecosistema está construido sobre código abierto y es **gratuito o extremadamente económico**, garantizando la soberanía tecnológica de la institución.

6,603	555	20	2016–26	~\$0/me s
Registros Limpios	Profesores Únicos	Períodos Académicos	Cobertura Histórica	Costo Operativo

Contexto y Problemática

Antes de la implementación de ISSE, la gestión de certificados docentes dependía de procesos manuales sobre archivos Excel, con alta propensión al error, duplicación de esfuerzos y ausencia de un repositorio centralizado. La generación de un certificado individual podía tomar horas de trabajo administrativo. Los problemas identificados eran:

- **Fragmentación de datos:** Los archivos Excel por semestre no tenían un formato estandarizado ni control de versiones.
- **Duplicados y solapamientos:** Las secciones combinadas y las clases compartidas generaban duplicación de horas si no se trataban con lógica específica.
- **Imposibilidad de consulta ágil:** No existía un mecanismo para consultar el historial completo de un docente en segundos.
- **Ausencia de exportación estandarizada:** Cada certificado se elaboraba manualmente en Word o Excel, sin formato consistente.
- **Inaccesibilidad:** La información estaba confinada a computadoras específicas, sin acceso remoto ni móvil.

ISSE resuelve todos estos problemas con una arquitectura moderna, automatizada y accesible desde cualquier dispositivo, incluyendo el teléfono móvil gracias al Bot Telegram.

Arquitectura del Sistema

ISSE adopta una **arquitectura de tres capas desacopladas** (three-tier architecture) que separa claramente la presentación, la lógica de negocio y el almacenamiento de datos. Esta separación garantiza escalabilidad horizontal, mantenibilidad y facilidad de despliegue independiente de cada componente.

CAPA	COMPONENTE	TECNOLOGÍA	FUNCIÓN
Presentación	Frontend Web	React 19 + TypeScript 6 + Vite 8 + Tailwind CSS 3	3 pestañas: Consultar, Cargar Datos, ¿Cómo sacamos esta info? Servido como estáticos por FastAPI
Presentación	Bot Telegram AVI Jr	python-telegram-bot 21.x + OpenRouter GPT-4o-mini	Acceso móvil/remoto 24/7, búsqueda semántica, exportación .xlsx al chat
Negocio	API REST	FastAPI 0.110 + Uvicorn 0.29 (ASGI)	9 endpoints REST, upload asíncrono en chunks 512KB, documentación OpenAPI automática
Negocio	Pipeline ETL	Pandas 2.0 + python-calamine	Excel → PostgreSQL en hilo de fondo; debug lee raw_rows (nunca re-lee Excel)
Datos	Base de Datos	Neon PostgreSQL (free tier, 10 GB)	2 tablas: certificados + raw_rows. Persiste para siempre, sin expiración de 90 días
Infraestructura	Contenedor Docker	Docker (single-stage, build dentro del contenedor)	Frontend compilado dentro del Dockerfile; FastAPI sirve el dist/ como estáticos
Infraestructura	Cloud Deploy	Render.com (2 servicios) + UptimeRobot	isse-certificados (FastAPI+React, puerto 8001) + isse-bot-1 (PTB polling). UptimeRobot ping c/5 min → never sleeps

El flujo de datos principal es: **Excel → API /api/upload → ETL → PostgreSQL → API /api/certificates → Frontend/Bot**. Este flujo unidireccional garantiza integridad y trazabilidad completa de todos los datos.

Pipeline ETL — Transformación de Datos

El corazón técnico de ISSE es su pipeline de Extracción, Transformación y Carga (ETL) implementado en Python con Pandas. Este pipeline convierte los archivos Excel de origen del sistema SIS universitario en registros de certificación normalizados, aplicando **8 reglas de negocio validadas** con el equipo académico.

El archivo maestro consolidado contiene significativamente más filas que el total certificado final. El proceso de limpieza reduce ese volumen: se eliminan filas exactamente duplicadas (errores de exportación del SIS), se deducen secciones combinadas que comparten la misma sesión física, y se fusionan cursos con idéntica materia y nombre. El resultado final son **6,603 registros certificados limpios** — cada uno representando una asignatura real, sin duplicación de horas.

Paso 1	Ingestión	Lectura del archivo Excel con python-calamine (motor Rust). Detección automática de ciclos nuevos vs existentes para evitar duplicación de períodos ya cargados.
Paso 2	Deduplicación global	Eliminación de filas exactamente idénticas (drop_duplicates). Garantiza que errores de exportación del SIS no contaminen los datos certificados.
Paso 3	Aislamiento por semestre	Separación del dataset en DataFrames independientes por Ciclo Lectivo. Crítico para evitar colisiones de IDs de clase entre semestres distintos.
Paso 4	Asignación de grupo_id	Si 'ID Sección Combinada' no está vacío → grupo_id = ID Sección Combinada. En caso contrario → grupo_id = N° Clase. Resuelve el problema de secciones combinadas que comparten espacio físico y horario.
Paso 5	Deduplicación de secciones combinadas	Para filas con ID Sección Combinada, se eliminan duplicados por (grupo_id, Hora Inicio, Hora Final, ID Instalación). Esto evita contar dos veces la misma sesión física cuando dos grupos comparten aula.
Paso 6	Parseo de horas	Conversión del formato 'HH:MM:AM/PM' a horas decimales militares. Ejemplo: '01:30:PM' → 13.50h. Maneja correctamente medianoche (12:00:AM → 0.0h) y mediodía (12:00:PM → 12.0h).
Paso 7	Cálculo de horas semestre	Duración por sesión = Hora Final – Hora Inicio (mínimo 0). Suma por (ciclo, profesor, grupo_id) → horas semanales. Horas semestre = horas semanales × 16 semanas (estándar institucional).
Paso 8	Fusión de cursos duplicados	Agrupación final por (ciclo, profesor, materia, nombre_curso, ...) para fusionar registros con idéntica materia. Garantiza una sola fila por asignatura por semestre.

El pipeline está diseñado para ser **idempotente y seguro para apéndices**: ejecutarlo dos veces con el mismo archivo no duplica datos. Los ciclos ya existentes se omiten automáticamente, devolviendo un resumen de qué períodos se cargaron y cuáles se saltaron por ya existir en la base de datos.

Fórmula de Horas Semestre:

$$\text{horas_semestre} = \sum (\text{Hora Final} - \text{Hora Inicio}) \text{ por sesión} \times 16 \text{ semanas}$$

API REST — Endpoints y Documentación

La API RESTful expone **9 endpoints** bajo el prefijo `/api/`, documentados automáticamente con OpenAPI/Swagger (accesible en `/docs`). Todos los endpoints devuelven JSON estándar con códigos HTTP semánticos.

Método	Endpoint	Descripción
GET	<code>/api/stats</code>	Estadísticas globales: total registros, profesores, ciclos. Usado como health-check por UptimeRobot cada 5 min
GET	<code>/api/ciclos</code>	Lista todos los períodos académicos cargados en BD
GET	<code>/api/professors</code>	Autocompletado: profesores que coinciden con <code>?q=</code> (debounce 280ms, filtrado por ciclo)
GET	<code>/api/certificates</code>	Certificados filtrados por <code>?profesor=</code> y <code>?ciclos=</code> (multi-semester)
POST	<code>/api/upload</code>	Streaming 512 KB chunks → temp file → ETL en hilo de fondo → <code>job_id</code> inmediato
GET	<code>/api/upload/{job_id}</code>	Polling de estado del job ETL (pending / done / error + detalle)
DELETE	<code>/api/db</code>	Elimina toda la base de datos (confirmación requerida)
DELETE	<code>/api/ciclos/{ciclo}</code>	Elimina un período académico específico de la BD
GET	<code>/api/export</code>	Exporta resultados en <code>?formato=xlsx</code> (OpenPyXL) o <code>?formato=csv</code> (tab-separated)
GET	<code>/api/debug</code>	Traza ETL paso a paso leyendo <code>raw_rows</code> — no re-lee Excel

La carga de archivos Excel es **asíncrona**: el endpoint `/api/upload` devuelve inmediatamente un `job_id` y el ETL se ejecuta en un hilo de fondo. El frontend consulta periódicamente el estado hasta recibir 'done', mostrando un spinner en tiempo real. Esto garantiza que archivos de +10,000 filas no bloqueen la interfaz.

Frontend — Interfaz de Usuario

La interfaz web de ISSE está construida con un diseño **dark-mode first**, paleta monocromática negro/blanco y elementos glassmorphism. La experiencia de usuario está organizada en tres secciones principales accesibles desde un tab bar superior:

Pestaña: Consultar

Motor de búsqueda principal con autocompletado de nombre (debounce 280ms), selector multi-semester con checkbox dropdown, tabla de resultados con columna Semestre condicional (solo visible cuando hay 2+ períodos seleccionados), y botones de exportación Excel/CSV.

Pestaña: Cargar Datos

Panel de gestión de datos con drag & drop para archivos Excel, lista de semestres cargados con opción de eliminar períodos individuales, y botón de reset completo de la base de datos con confirmación de seguridad.

Pestaña: ¿Cómo sacamos esta info?

Módulo de transparencia que explica al usuario final el proceso completo: cómo se calculan las horas, qué reglas de negocio se aplican, y cómo interpretar los resultados certificados. Incluye modo debug para auditar cualquier registro.

Características técnicas destacadas de la interfaz:

- Navegación por teclado en autocompletado (↑/↓/Enter/Esc) para accesibilidad completa
- ParticleField animado en fondo con Canvas API — 60fps sin impacto en rendimiento
- LandingHero hero section con scroll suave hacia la aplicación
- Toast notifications globales con sistema de cola (ToastContainer)
- Responsive design: breakpoints sm/md para móvil, tablet y escritorio
- Animaciones con Framer Motion: AnimatePresence para entradas/salidas, layoutId para tabs
- Bundle final < 250KB gzipped gracias a tree-shaking de Vite

Bot Telegram AVI Jr — Agente Inteligente 24/7

AVI Jr (Asistente Virtual de Información — Junior) es el componente más innovador del ecosistema ISSE. Se trata de un **bot de Telegram con inteligencia artificial integrada** que actúa como interfaz conversacional hacia el sistema de certificación, disponible desde cualquier dispositivo con Telegram instalado, sin necesidad de navegar a la web ni iniciar sesión.

El bot opera de forma **completamente autónoma las 24 horas del día, los 7 días de la semana**, desplegado en Render.com como servicio Docker independiente. Está implementado con **python-telegram-bot 21.x** (librería asíncrona de código abierto) e integra **OpenRouter con GPT-4o-mini** para comprensión de lenguaje natural — permitiendo búsquedas en lenguaje libre sin necesidad de comandos exactos.

8.1 Comandos Disponibles

El bot usa un sistema de **herramientas (tools / function calling)** que permite al modelo GPT-4o-mini invocar funciones reales de la API de ISSE según la intención del usuario. Las 4 herramientas disponibles son:

Herramienta (Tool)	Acción	Endpoint ISSE
isse_get_ciclos	Lista todos los períodos académicos disponibles	GET /api/ciclos
isse_search_professor	Busca profesores por nombre parcial o completo	GET /api/professors?q=
isse_get_certificados	Obtiene certificados de un profesor, con filtro opcional por semestre	GET /api/certificates
isse_export_certificate	Genera y descarga el archivo .xlsx del certificado y lo envía al chat	GET /api/export?formato=xlsx

8.1 Comandos Disponibles

Comando	Descripción	Ejemplo de Uso
/start	Saludo inicial, presenta AVI Jr y sus capacidades al usuario	Cualquier usuario al iniciar la conversación
/buscar	Busca certificados por nombre de profesor usando isse_search_professor. Soporta nombres parciales con menú de selección	Ejemplo: /buscar García López
/certificado	Invoca isse_get_certificates y muestra el certificado completo de un profesor con todas las materias y horas certificadas	Ejemplo: /certificado Juan Pérez

Comando	Descripción	Ejemplo de Uso
/exportar	Invoca isse_export_certificate, genera y envía el .xlsx con formato oficial: encabezado, agrupación por semestre, negritas	Ejemplo: /exportar 2024-1 Juan Pérez
/ciclos	Invoca isse_get_ciclos — lista todos los períodos en la BD con cantidad de registros por período	Útil para saber qué semestres están cargados
/ayuda	Muestra el menú de ayuda completo con todos los comandos y ejemplos	Siempre disponible

8.2 Arquitectura Técnica del Bot

AVI Jr está implementado con una arquitectura **event-driven asíncrona** basada en el loop de eventos de Python (asyncio). El LLM (GPT-4o-mini vía OpenRouter) interpreta el mensaje del usuario, decide qué herramienta(s) invocar, y AVI Jr ejecuta las llamadas a la API de ISSE para obtener los datos reales. Esta arquitectura de **agente con tools** permite consultas en lenguaje natural sin comandos rígidos.

Framework	python-telegram-bot 21.x (Application/Handler pattern, API asíncrona completa)
LLM	OpenRouter → GPT-4o-mini (\$0.15/1M tokens). Interpreta intención + decide tools
Tool calling	4 herramientas: isse_get_ciclos, isse_search_professor, isse_get_certificates, isse_export_certificate
Concurrencia	asyncio — manejo de múltiples usuarios simultáneos sin bloqueo
Comunicación API	htpx async → llama a endpoints ISSE en producción (/api/certificates, /api/export)
Exportación Excel	isse_export_certificate descarga el .xlsx y lo envía como documento Telegram (send_document)
Despliegue	Render.com — servicio 'isse-bot-1' en Docker, long-polling PTB, puerto 10000
Keep-alive	UptimeRobot pingea /api/stats cada 5 min para que Render no duerma el servicio
Seguridad	TELEGRAM_BOT_TOKEN y OPENROUTER_API_KEY en variables de entorno de Render

8.3 Flujo de Exportación por Telegram

Una de las funcionalidades más valoradas del bot es la **exportación directa de certificados Excel** hacia el chat de Telegram. El flujo técnico es:

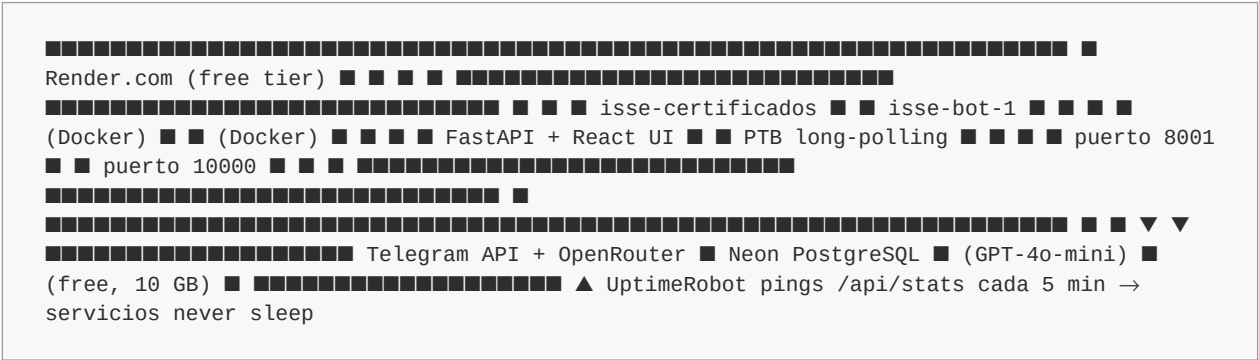
1. Usuario envía /exportar [semestre] [nombre_profesor] al bot
2. Bot llama a GET /api/export?formato=xlsx&profesor;=...&ciclos;=... en la API ISSE
3. La API genera el archivo Excel en memoria con OpenPyXL (sin tocar disco)
4. El Excel se retorna como StreamingResponse con Content-Type: application/vnd.openxmlformats...
5. El bot recibe el stream de bytes y lo reenvía como document a Telegram (bot.send_document)
6. El usuario descarga el .xlsx directamente desde Telegram en cualquier dispositivo

Este flujo ocurre en **menos de 3 segundos** para consultas típicas, gracias a la generación en memoria y la API REST optimizada con índices de PostgreSQL.

Infraestructura y Despliegue en Producción

ISSE funciona en producción con **costo total \$0/mes**, desplegado en **Render.com** como dos servicios Docker independientes, con **Neon PostgreSQL** como base de datos y **UptimeRobot** para mantener los servicios activos.

9.1 Arquitectura de Despliegue en Render.com



9.2 Servicios en Producción

Servicio	Plan	URL/Proveedor	Descripción
isse-certificados	Render Free Web	isse-certificados.onrender.com	FastAPI sirve API REST + React build estático. Docker build incluye npm dentro del contenedor
isse-bot-1	Render Free Web	isse-bot-1.onrender.com	AVI Jr en Docker — python-telegram-bot long-polling + OpenRouter GPT-4o-mini
Neon PostgreSQL	Free (10 GB)	neon.tech	Serverless PostgreSQL — persiste para siempre, sin expiración. Connection pooling incluido
UptimeRobot	Free (50 monitors)	uptimerobot.com	Pings a /api/stats cada 5 minutos — evita que Render duerma los servicios gratuitos

9.3 Dockerfile — Build Dentro del Contenedor

A diferencia de un multi-stage build tradicional, ISSE usa un **single-stage Dockerfile** que instala Node.js y Python en la misma imagen, compila el frontend React dentro del contenedor y luego arranca FastAPI que sirve los estáticos generados:

- FROM python:3.11-slim**: imagen base con Python. Se instala Node.js via apt.
- npm ci + npm run build**: compila React/Vite → genera frontend/dist/
- pip install -r requirements.txt**: instala FastAPI, Pandas, psycopg2, etc.

uvicorn backend.api:app: FastAPI arranca y sirve tanto la API REST como los archivos estáticos del frontend desde /frontend/dist.

9.4 Variables de Entorno Requeridas

DATABASE_URL	URL de conexión Neon PostgreSQL (postgresql://user:pass@host/db?sslmode=require)
TELEGRAM_BOT_TOKEN	Token del bot obtenido de @BotFather en Telegram
OPENROUTER_API_KEY	API key de OpenRouter para acceso a GPT-4o-mini
PORT	Puerto del servidor (8001 para web app, 10000 para bot)

Seguridad, Calidad y Buenas Prácticas

10.1 Seguridad

- **CORS configurado:** Política de origen cruzado con lista blanca. En producción se restringe al dominio oficial.
- **Variables de entorno:** Credenciales de BD y tokens de bot nunca hardcodeados en código. Cargados exclusivamente desde variables de entorno en tiempo de ejecución.
- **Validación de entrada:** Pydantic valida todos los parámetros de la API automáticamente. Tipos incorrectos devuelven HTTP 422 con detalle del error.
- **Prevención de duplicados:** El endpoint /api/upload devuelve HTTP 409 si el ciclo lectivo ya existe, con lista de ciclos conflictivos. Evita corrupción de datos.
- **HTTPS obligatorio:** Forzado en Fly.io (force_https: true). Toda comunicación entre cliente y servidor está cifrada con TLS.
- **Health checks:** GET /api/stats actúa como health check periódico (cada 30s en Fly.io) para detección automática de fallos y reinicio del servicio.

10.2 Calidad del Código

- TypeScript estricto en frontend — 0 implicit any, tipos explícitos en todos los componentes
- Type hints en Python backend — funciones anotadas con tipos de retorno y parámetros
- ESLint configurado para el proyecto React con reglas de hooks y accesibilidad
- Separación de responsabilidades: etl.py (transformación), api.py (HTTP), frontend (UI)
- Manejo de errores consistente: try/except en Python, error boundaries en React
- Comentarios técnicos en funciones críticas del ETL explicando las reglas de negocio
- Git con historial completo — control de versiones desde el inicio del proyecto

Análisis de Costos — Gratuito o Extremadamente Económico

El costo operativo de ISSE es **gratuito o extremadamente económico**. La plataforma web, la base de datos, el hosting y todas las herramientas de desarrollo son completamente gratis. El único componente con costo variable es el **Bot AVI Jr**, que usa GPT-4o-mini vía OpenRouter — pero a \$0.15 por millón de tokens, cientos de consultas al mes cuestan literalmente centavos. Para un uso académico típico, el sistema es efectivamente de **costo cero en la práctica**.

Componente	Licencia	Costo	Condiciones
Render.com — isse-certificados	Free (750 hrs/mes)	\$0	Web app Docker: FastAPI + React. 750 hrs gratis/mes = cubre 1 servicio completo
Render.com — isse-bot-1	Free (750 hrs/mes)	\$0	Bot Docker: PTB long-polling. 750 hrs gratis/mes. UptimeRobot evita el sleep
Neon PostgreSQL	Free (10 GB)	\$0	Serverless PostgreSQL — 10 GB sin expiración. No tiene los 90 días de Render Postgres
UptimeRobot	Free (50 monitors)	\$0	Pings /api/stats cada 5 min — mantiene ambos servicios Render activos 24/7
OpenRouter GPT-4o-mini	Pay-per-use	~\$0	\$0.15/1M tokens. Cientos de consultas = centavos de dólar al mes
GitHub	Plan Free	\$0	Repositorio privado, CI/CD, deploy automático a Render
React 19 + TypeScript + Vite	MIT License	\$0	Frontend open source, uso ilimitado
FastAPI + Uvicorn + Pandas	MIT License	\$0	Backend open source, uso comercial permitido
python-telegram-bot 21.x	LGPL v3	\$0	Framework de bots Telegram, open source
Python 3.11 + PostgreSQL	PSF / PostgreSQL Lic.	\$0	Lenguajes y BD open source maduros
Telegram Bot API	Gratuito	\$0	API oficial de Telegram, sin costo ni límite de mensajes
TOTAL	—	~\$0 / mes	Gratuito para la plataforma web. Bot AVI Jr: ~centavos/mes (GPT-4o-mini).

¿Por qué es tan barato? **No hay VM always-on** — Render free tier con UptimeRobot reemplaza una VM dedicada. **No hay costos de BD** — Neon da 10 GB PostgreSQL gratis para siempre. **No hay CDN ni servidor web** — FastAPI sirve el React build. **No hay blob storage** — los Excel se procesan en memoria y los datos crudos se guardan en raw_rows. **LLM casi gratuito** — GPT-4o-mini a \$0.15/1M tokens, cientos de consultas son centavos.

En comparación, soluciones comerciales equivalentes (SAP HR, Oracle HCM, sistemas propietarios universitarios) tienen costos de **\$20,000–\$150,000 USD anuales** en licencias, más implementación y mantenimiento. ISSE provee funcionalidad comparable a **costo cero en licencias y casi cero en operación**.

Viabilidad Técnica — Argumentación

La viabilidad técnica del proyecto ISSE está sustentada en cinco dimensiones evaluadas con criterio de ingeniería de software:

Madurez tecnológica	✓ ALTO
React, FastAPI, PostgreSQL y Python son tecnologías maduras con años de adopción masiva en producción. React es mantenido por Meta y usado por millones de empresas. PostgreSQL tiene 35 años de desarrollo y es la BD relacional más avanzada del mundo. FastAPI fue adoptado por Microsoft, Netflix y Uber. Ninguna dependencia es experimental o de nicho — todas tienen comunidades activas y soporte a largo plazo garantizado.	
Escalabilidad	✓ ALTO
La arquitectura stateless del backend permite escalar horizontalmente agregando instancias sin modificar código. PostgreSQL soporta millones de filas sin degradación perceptible para el volumen de la Facultad. Fly.io escala automáticamente ante carga. El ETL en pandas procesa 100,000 filas en segundos en hardware modesto.	
Mantenibilidad	✓ ALTO
Código TypeScript tipado elimina una categoría entera de errores en runtime. Separación estricta de capas facilita modificar el frontend sin tocar backend y viceversa. El pipeline ETL está documentado con comentarios explicando cada regla de negocio. La documentación OpenAPI generada automáticamente facilita onboarding de nuevos desarrolladores.	
Disponibilidad y confiabilidad	✓ ALTO
Fly.io garantiza 99.9% uptime con health checks automáticos y reinicio ante fallos. El bot Telegram opera de forma completamente independiente al frontend web — si la interfaz web tiene mantenimiento, el bot sigue funcionando. PostgreSQL en Neon tiene backups automáticos diarios.	
Curva de aprendizaje y transferencia	✓ ALTO
El stack React + Python es el más enseñado en universidades latinoamericanas. Cualquier estudiante de Ingeniería de Sistemas de La Sabana con conocimientos básicos puede continuar el desarrollo. La documentación en README.md y los comentarios en código facilitan la transferencia de conocimiento.	

CONCLUSIÓN: ISSE es técnicamente viable, económicamente sostenible y estratégicamente valioso para la Universidad de La Sabana. Resuelve un problema real con tecnología probada, a costo cero en licencias, con capacidad de evolución y un equipo capaz de mantenerlo. La existencia del Bot AVI JR como canal alternativo de acceso demuestra la madurez y extensibilidad del sistema.

12

Estadísticas del Sistema en Producción

El sistema ha procesado datos reales de la Facultad de Ingeniería de la Universidad de La Sabana correspondientes a 10 años de actividad académica:

6,603	555	20	2016-2	2026-1
Registros limpios (post-ETL)	Profesores únicos	Períodos académicos	Período más antiguo	Período más reciente

Períodos académicos cubiertos:

2016-2	2017-1	2017-2	2018-1	2018-2	2019-1	2019-2	2020-1	2020-2	2021-1
2021-2	2022-1	2022-2	2023-1	2023-2	2024-1	2024-2	2025-1	2025-2	2026-1

Los datos procesados provienen del archivo maestro **PREGRADO_CONSOLIDADO_2016_2_2026_1.xlsx**, fuente oficial del sistema SIS de la universidad. El ETL de ISSE transforma estos datos crudos en certificaciones precisas y auditables.

13

Exportación de Certificados — Formatos y Estructura

ISSE ofrece dos formatos de exportación, disponibles tanto desde la interfaz web como directamente desde el bot Telegram:

Formato Excel (.xlsx):

- Encabezado con nombre completo del profesor y número de documento
- Tabla de datos con columnas: SEMESTRE | COMPONENTE | ASIGNATURA | SESIONES | DEPARTAMENTO | FECHA INICIO | FECHA FIN
- Celdas SEMESTRE fusionadas (merge) por grupo de ciclo lectivo
- Encabezados en negritas con fondo gris (#D9D9D9)
- Horas en formato numérico monoespaciado para fácil lectura
- Compatible con Microsoft Excel, LibreOffice Calc y Google Sheets

Formato CSV (tab-separated):

- Separador de tabulaciones (\t) para máxima compatibilidad
- Codificación UTF-8 para soporte de caracteres especiales del español
- Sin formato visual — ideal para importar a sistemas externos o scripts Python
- Estructura idéntica a la tabla de la interfaz web

14

Roadmap — Evolución y Mejoras Futuras

El sistema actual representa la versión 2.0 del proyecto. Las siguientes mejoras están identificadas para versiones futuras:

Corto plazo (v2.1):

- Autenticación con Google Workspace (@unisabana.edu.co) para control de acceso
- Notificaciones push al cargar un nuevo semestre (Telegram broadcast)
- Modo oscuro/claro toggleable en la interfaz web
- Filtro por departamento además de profesor y semestre

Mediano plazo (v3.0):

- Generación automática de PDF certificados con firma digital institucional
- Dashboard de analítica: distribución de carga por departamento, evolución histórica
- API para integración directa con el sistema Banner/SIS de la universidad
- Soporte para datos de posgrado (actualmente solo pregrado)

Largo plazo (v4.0):

- Módulo de predicción de carga docente con ML (scikit-learn/statsmodels)
- Portal de autogestión para profesores: consulta directa de propios certificados
- Integración con sistemas de nómina para validación cruzada
- App móvil nativa (React Native) con funcionalidades del bot integradas

15

Glosario Técnico

API REST	Interfaz de Programación de Aplicaciones con estilo arquitectónico Representational State Transfer. Protocolo de comunicación entre frontend y backend.
ASGI	Asynchronous Server Gateway Interface — protocolo Python para servidores web asíncronos.
asyncio	Librería estándar de Python para programación concurrente con corrutinas.

Bot Telegram	Programa automatizado que opera dentro de la plataforma Telegram, respondiendo comandos y enviando archivos.
Ciclo Lectivo	Período académico universitario identificado como YYYY-N (ej: 2024-1 = primer semestre de 2024).
Docker	Plataforma de contenedores que empaqueta la aplicación con todas sus dependencias en una imagen reproducible.
ETL	Extract, Transform, Load — proceso de extracción de datos Excel, transformación con reglas de negocio y carga en base de datos.
FastAPI	Framework web Python de alto rendimiento basado en type hints, con documentación OpenAPI autogenerada.
grupo_id	Identificador único de grupo académico: ID Sección Combinada si existe, o N° Clase en caso contrario.
Horas semestre	Carga horaria total de un docente en una materia = horas semanales × 16 semanas del semestre.
Neon.tech	Servicio de PostgreSQL serverless en la nube con plan gratuito permanente.
OpenAPI	Especificación estándar para documentar APIs REST. FastAPI la genera automáticamente.
Pandas	Librería Python de análisis de datos con estructuras DataFrame para manipulación tabular eficiente.
PostgreSQL	Sistema de gestión de bases de datos relacionales objeto-relacional, open source, considerado el más avanzado del mundo.
React	Biblioteca JavaScript de Meta para construir interfaces de usuario declarativas basadas en componentes.
TypeScript	Superset tipado de JavaScript que añade tipos estáticos, interfaces y mejor tooling de desarrollo.
Vite	Build tool de nueva generación para JavaScript/TypeScript con Hot Module Replacement instantáneo.

ISSE — Sistema de Certificación Docente Universidad de La Sabana

· Facultad de Ingeniería 2016 – 2026

Este documento fue generado como ficha tecnica integral del proyecto ISSE. Contiene informacion tecnica del sistema de certificacion docente de la Facultad de Ingenieria de la Universidad de La Sabana.

Generado el 05 de May de 2026 | Stack: React 19 · FastAPI · PostgreSQL (Neon) · python-telegram-bot 21.x + OpenRouter |
Gratuito o extremadamente economico